

National University of Computer and Emerging Sciences



Laboratory Manual

for

Data Structures Lab

Course Instructor	Mr. Muhammad Naveed
Lab Instructor	Mr. Durraiz Waseem
Semester	Fall 2025

Department of Computer Science

FAST-NU, Lahore, Pakistan

Problem 1: Heap Trees:

1. In this task, you are going to implement a class for Max Heap. Each node of this Max Heap will contain the roll number, and CGPA of a student. The heap will be organized on the basis of students'

```
class StudentMaxHeap;
class Student {
friend class StudentMaxHeap;
private: int rollNo; // Student's roll number
        double cgpa; // Student's CGPA };

class StudentMaxHeap {
private: Student* st; // Array of students which will be arranged like a MaxHeap
        int currSize; // Current number of students present in the heap
        int maxSize; // Maximum no. of students that can be present in heap
public: StudentMaxHeap (int size); // Constructor
        ~StudentMaxHeap(); // Destructor
        bool isEmpty(); // Checks whether the heap is empty or not bool
        isFull(); // Checks whether the heap is full or not
};
```

CGPAs i.e. the student having the maximum CGPA will be at the root of the heap. The class definitions will look like above.

2. Implement a public member function of the StudentMaxHeap class which inserts the record of a new student (with the given roll number and CGPA) in the Max Heap. The prototype of your function should be:

bool insert (int rollNo, double cgpa);

This function should return true if the record was successfully inserted in the heap and it should return false otherwise. The worst-case time complexity of this function should be ($\lg n$). If two students have the same CGPA then their records should be stored in a way such that at the time of removal if two (or more) students have the same highest CGPA then the student with smaller roll number should be removed before the students with larger roll numbers.

3. Now, implement a public member function to remove that student's record from the Max Heap which has the highest CGPA. The prototype of your function should be:

bool removeBestStudent (int& rollNo, double& cgpa);

Before removing the student's record, this function will store the roll number and CGPA of the removed student in its two reference parameters. It should return true if the removal was successful and it should return false otherwise. The worst-case time complexity of this function should also be ($\lg n$).